QUBE

QUEUE-BASED UNIFIED BALANCING ENGINE

Name: Prabeen Kumar Swain

ID: B125082

Branch: CSE-B

College: International Institute of Information Technology, Bhubaneswar

1. Abstract

QUBE (Queue-Based Unified Balancing Engine) is a C++17-based queue management and service-flow simulation system designed to investigate the trade-offs between service efficiency, priority-based scheduling, and fairness. It models a service environment in which customers arrive with different priority levels, wait in separate queues, and are assigned to available service counters according to configurable scheduling policies.

The system maintains three independent FIFO queues for P1, P2, and P3 customers, where P3 has the highest priority, followed by P2 and P1. Customers remain in their respective queues while the scheduling engine selects eligible customers for service. Four service counters have configurable primary priority assignments, enabling the simulation of dedicated service capacity and cross-class counter sharing.

QUBE supports First-Come, First-Served (FCFS), Priority Scheduling, Shortest Job First (SJF), and Aging-based scheduling. Its aging mechanism helps address starvation by allowing sufficiently long-waiting customers to override normal counter preferences under the aging policy. A raylib-based graphical interface presents an isometric waiting room, moving customers, service counters, and performance statistics. The system measures waiting time, throughput, counter utilization, starvation, and fairness-related indicators, providing an experimental platform for comparing service-allocation strategies.

2. Project Description

In multi-class service environments, customers may have different urgency levels. A scheduler must serve high-priority customers efficiently while limiting excessive delays and starvation among lower-priority customers. QUBE separates queue organization from scheduling decisions.

Objectives

- Implement and compare multiple scheduling policies under the same simulated workload.
- Maintain independent FIFO queues for P1, P2, and P3 without globally rearranging customers.
- Support four service counters with configurable primary priority assignments and counter borrowing.
- Explore starvation prevention through aging-based scheduling.
- Visualize customer movement from arrival through queueing, service, and completion.
- Measure service efficiency, counter utilization, waiting-time distribution, and fairness indicators.

Core Components

Component	Responsibility
Customer management	Tracks customer ID, priority, arrival time, service duration, and service state.
Three priority queues	Maintains separate FIFO lanes for P3 (highest), P2, and P1 (lowest).
Scheduling engine	Implements FCFS, Priority, SJF, and Aging-based selection.
Counter allocation	Uses four counters with default assignments P3, P2, P1, and P1; idle counters may borrow from other classes according to policy.
Visualization	Uses raylib to render the isometric waiting area, queues, counters, and customer activity.
Performance analytics	Tracks average and maximum wait, throughput, utilization, starvation, Jain fairness index, waiting-time spread, and counter balance.

Scheduling Policies

FCFS — Selects the oldest waiting customer across the queue fronts, without enforcing counter-class preference.

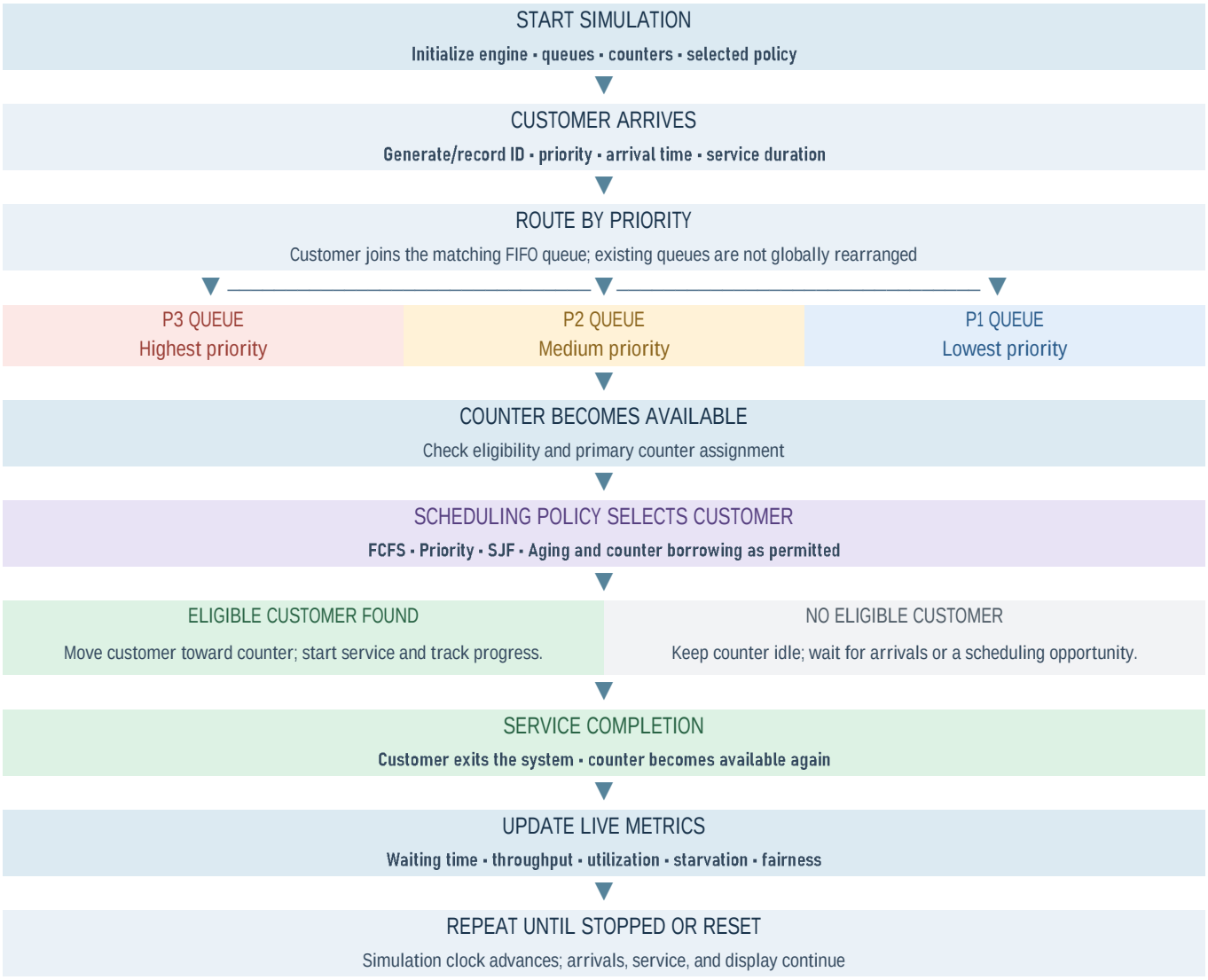
Priority — Prefers the counter's assigned queue; when it is empty, borrows according to priority class.

SJF — Selects the waiting customer with the shortest service duration, without enforcing counter-class preference.

Aging — Prefers the counter's assigned queue, borrows using aging scores, and allows sufficiently long-waiting queue-front customers to override the normal preference.

3. System Workflow — Workflow Tree

The tree below shows the main execution path and the parallel decisions that occur during customer arrival, queue assignment, scheduling, counter allocation, and performance monitoring.



Workflow Summary

1. Initialize the simulation and choose a scheduling policy.
2. Register each arriving customer and route them to the P1, P2, or P3 FIFO queue.
3. When a counter becomes available, inspect eligible queue-front customers and apply the selected policy.
4. Apply counter-class preferences, borrowing rules, and aging overrides where the selected policy supports them.
5. Animate the customer approaching and receiving service; mark the customer completed when service ends.
6. Update performance metrics and repeat as simulated time advances.

4. Technology Stack

Technology	Purpose
C++17	Scheduling engine and simulation logic
raylib 5.5	Graphical rendering and interaction
CMake	Build configuration and dependency management
GCC/MinGW or MSVC	C++ compilation on Windows
Automated C++ tests	Validation of queueing, scheduling, timing, aging, and counter behaviour

5. Performance Indicators

Average waiting time: Mean time customers wait before service begins.

Maximum waiting time: Longest observed wait in the workload.

Throughput: Number of completed customers per unit of simulated time.

Counter utilization: Fraction of simulated time counters are busy or otherwise occupied, according to engine definitions.

Starvation count: Customers exceeding the configured waiting-time threshold.

Jain fairness index: Equality indicator for the metric being evaluated; interpret alongside absolute wait times.

90th-percentile wait: Waiting-time level below which 90% of observed customers fall.

Counter balance: How evenly work is distributed across counters.

6. Implementation Status and Scope

The project source and documentation describe three priority queues, four configurable counters with default assignments P3/P2/P1/P1, FCFS, Priority, SJF and Aging policies, counter borrowing, performance statistics, a raylib-based isometric visualization, and automated engine tests. Some visual refinements and configurable counter-count options are listed as future work. This document summarizes the code and project documentation; it does not claim an independent runtime build or execution verification.

7. Short Project Summary

QUBE is a graphical queue-management simulator developed in C++17 that explores priority-based scheduling, aging, and adaptive service-counter allocation. By maintaining separate queues for P1, P2, and P3 customers and supporting four service counters, it enables comparison of FCFS, Priority, SJF, and Aging policies. Its isometric visualization and performance metrics provide an interactive environment for studying waiting time, throughput, starvation, resource utilization, and fairness in multi-class service systems.